

MÓDULO 4 • PLENARIA P4

Del sistema que funciona al sistema que se defiende

PRUEBAS • DESPLIEGUE PRODUCTIVO • DOCUMENTO FINAL • DEFENSA

Martes 29 de septiembre • 19:00 – 20:30

Diplomado en Desarrollo Web y Aplicaciones Móviles • UAJMS

M.Sc. Ing. Isaac Lange Aguilar • Gestión 2026

Noventa minutos, seis momentos

1	Lo que dejó el E2 Lo que se vio en la primera vertical desplegada	10 min
2	Pruebas con evidencia De criterio de aceptación a caso de prueba, y la evidencia que vale	25 min
3	Del despliegue al despliegue verificado Pipeline, monitoreo y rendimiento medido	15 min
4	Documento final y manuales Seguridad, pruebas, despliegue, Capítulo 3 y anexos	15 min
5	Protocolo y rúbrica de la defensa Doce minutos por persona, sobre el sistema en producción	15 min
6	Lo que hay que dejar listo E3 del sábado 3, E4 del sábado 10 y la defensa	10 min

BLOQUE 1 • LECTURA CRÍTICA

Lo que dejó el E2

La primera vertical en producción: lo que se repite y lo que ya funciona.

01

Cinco fallas que se repitieron en el E2

LO QUE MOSTRARON LAS 28 REVISIONES • SE CORRIGE ANTES DEL E3

Falla	Cómo se vio en el E2	Qué hacer antes del sábado 3
Nada que revisar en producción	10 de 28: cuatro sin entrega, cuatro sin URL pública y dos con solo el login. Se aplica el tope de 50.	Desplegar primero y programar después: sin URL, el E3 vuelve a tener tope
Salud ausente o fuera del contrato	7 de 28 sin GET /api/v1/salud: "No aplica", "pendiente", /health o /v1/salud	Con API propia, la ruta con SELECT 1; con Supabase, una función RPC salud. URL completa en la Tabla 1
Credenciales reales o débiles	9 de 28: correos personales o de gmail.com, admin@admin.com, 123456, password123. Dos contraseñas personales en un PDF que queda en Moodle	Una cuenta @proyecto.test por rol, contraseña de 10+. Si expuso una contraseña real, cambiarla hoy
Restos de plantilla y de asistente	13 de 28: corchetes, "POR CONFIRMAR", "***", la hoja "Verificación previa" y un texto que "no verifica" el propio sistema (-5)	El autor sabe lo que hace su sistema: escribirlo. Buscar [, ** y "Verificación" antes de exportar
El Capítulo 2 no es lo desplegado	8 sin 2.6 escrito, 7 con un contrato de rutas distintas a las reales y 3 con sistema en línea pero sin documento subido	2.6 con lo construido, una decisión y una dificultad. Contrato y diccionario copiados de lo que corre

Cuatro señales de una vertical sana

LO QUE YA SE VIO EN LAS MEJORES ENTREGAS DEL GRUPO

SEÑAL

Salud que consulta la base

Responde 200 solo si la base contesta. Un chequeo en verde que no toca la base no prueba que el sistema funcione.

SEÑAL

La regla vive en el servidor

Reserva con FOR UPDATE, stock y traslado en una transacción, vencimiento calculado en la API: la pantalla no decide.

SEÑAL

Una dificultad de verdad

El turno de 19:00 a 07:00 que la validación rechazaba; el sensor con una primera lectura falsa. Contada con su resolución.

SEÑAL

Una cuenta ficticia por rol

Correos .test, una cuenta por rol: quien revisa entra y prueba el 403 sin pedir nada a nadie.

Esta semana se suma lo que convierte una demo en un sistema: roles verificados en el servidor y pruebas con evidencia.

BLOQUE 2 • FUNDAMENTO

Pruebas con evidencia

Un requisito sin prueba es una promesa. Una prueba sin evidencia no existe.

02

Del criterio de aceptación al caso de prueba

APARTADO 2.8 • CADA DADO / CUANDO / ENTONCES SE CONVIERTE EN UN CASO

	CRITERIO		CASO DE PRUEBA
RF-03 • camino feliz	Dado Recepción, cuando registra un pedido válido, entonces queda PENDIENTE	→	CP-05 • POST /pedidos válido → 201 y estado PENDIENTE en la base
RF-03 • error	Dado un pedido sin ítems, cuando se envía, entonces se rechaza con mensaje	→	CP-06 • POST /pedidos con items vacío → 400 y el campo items marcado
RF-05 • restricción de rol	Dado Recepción, cuando intenta marcar un pedido como Listo, entonces no puede	→	CP-09 • PATCH /pedidos/7/estado con token de Recepción → 403
RNF-01 • rendimiento	La lista de pedidos responde en menos de 2 s con 100 registros	→	CP-14 • GET /pedidos con 100 registros en producción → 0,8 s medidos

Si un criterio no se puede convertir en caso de prueba, el criterio está mal escrito.

La tabla de casos de prueba

LINEAMIENTOS 3.2.8 • ID, ESCENARIO, ESPERADO, OBTENIDO Y ESTADO

ID	Escenario	Esperado	Obtenido	Estado
CP-01	Login válido (RF-01)	200 y token con vencimiento	200, token de 60 min • Fig. 21	Aprobado
CP-02	Contraseña incorrecta (RF-01)	401, mensaje genérico	401 "Correo o contraseña incorrectos"	Aprobado
CP-06	Pedido sin ítems (RF-03)	400 y campo ítems marcado	500 sin mensaje → corregido → 400	Aprobado tras corrección
CP-09	Recepción marca Listo (RF-05)	403	403 • reporte Newman del 30/09	Aprobado
CP-11	Entregar lo ya entregado (RF-06)	409	200: se duplicó la entrega	Fallido • issue #14
CP-14	Lista de 100 registros (RNF-01)	Menos de 2 s en producción	0,8 s medidos en producción	Aprobado

Un caso fallido registrado vale más que una tabla donde todo "pasó".

La columna "obtenido" dice lo que se observó (código, mensaje, tiempo) y dónde está la evidencia.

Cuatro niveles, lo mínimo de cada uno

NO SE PIDE COBERTURA TOTAL: SE PIDE QUE CADA NIVEL TENGA ALGO

Nivel	Mínimo para el E3	Herramientas (ejemplos)
Unitaria	Dos o tres funciones con reglas de negocio: totales, validaciones, transiciones de estado	Jest, Vitest, pytest, JUnit, flutter_test
Integración (API + base)	Cada ruta Must con camino feliz y error, incluidos 401 y 403	Supertest, Postman con Newman, pytest con httpx
Funcional (de punta a punta)	El flujo Must completo desde la interfaz, en producción, con capturas fechadas	Guion manual, o Playwright / Cypress si ya lo usan
Rendimiento	Una medición del RNF de tiempo sobre la URL pública	Herramientas del navegador, k6, Artillery, Postman

Automatizar las pruebas de la API es lo que más rinde por hora invertida.

Una colección de Postman con aserciones, ejecutada con Newman y con su reporte en el repositorio, ya es prueba automatizada.

Una prueba automatizada que se entiende

INTEGRACIÓN • LA RUTA PROTEGIDA DEVUELVE 403 AL ROL EQUIVOCADO

```
// test/pedidos.test.js • Jest + Supertest
test('Recepción no puede marcar Listo', async () => {
  const token = await login('recepcion@demo.bo');
  const res = await request(app)
    .patch('/api/v1/pedidos/7/estado')
    .set('Authorization', `Bearer ${token}`)
    .send({ estado: 'LISTO' });
  expect(res.status).toBe(403);
  expect(res.body.error.codigo).toBe('ROL_NO_PERMITIDO');
});
// npx jest --json > evidencia/jest-2026-10-01.json
```

Línea	Qué hace	Si falla	Caso
<code>login()</code>	Pide un token real	No llega a la ruta	CP-01
<code>.set()</code>	Envía el token Bearer	401 en vez de 403	CP-08
<code>.send()</code>	Cuerpo válido	Un 400 tapa el 403	CP-09
<code>toBe(403)</code>	Verifica el código	La ruta no protege	CP-09
<code>error.codigo</code>	Verifica el formato	El front no sabe qué mostrar	CP-09

La salida del runner, guardada en el repositorio con fecha, es la evidencia.

Si el documento dice "solo el administrador", existe una prueba que muestra el 403.

La evidencia que vale

SI NO SE PUEDE VERIFICAR, NO ES EVIDENCIA

VALE

Captura fechada en producción

Muestra la URL pública, la fecha y el resultado, con datos ficticios. Numerada como figura.

VALE

Reporte del runner en el repositorio

Salida de Jest, pytest o Newman en /evidencia, con fecha. Se puede volver a ejecutar.

VALE

El fallo con su corrección

Caso fallido, commit que lo corrige y reejecución. Muestra que se probó de verdad.

VALE

La medición del RNF

Tiempo medido en producción, con la herramienta y la cantidad de registros declaradas.

No vale: "funciona", capturas de localhost, recortes sin URL ni fecha.

BLOQUE 3 • FUNDAMENTO

Del despliegue al despliegue verificado

03

Pipeline, monitoreo y rendimiento medido: lo que convierte una URL en un sistema.

Un pipeline mínimo de integración y despliegue

GITHUB ACTIONS, GITLAB CI O EL DESPLIEGUE AUTOMÁTICO DE SU PLATAFORMA



Un archivo de workflow y la integración de su plataforma bastan: no se pide infraestructura propia.

Sin pipeline, el procedimiento manual se documenta paso a paso en 2.9. El pipeline suma; su ausencia documentada no resta.

Monitoreo mínimo en producción

LO QUE SE CONFIGURA UNA VEZ Y AVISA SOLO

Qué	Cómo	Ejemplos
Disponibilidad	Un servicio externo consulta /api/v1/salud cada pocos minutos y avisa si falla	UptimeRobot, Better Stack, cron-job.org
Errores	Cada 500 queda registrado con ruta, hora y traza, sin datos sensibles	Sentry, logs de la plataforma
Tiempos de respuesta	Se mide la ruta del RNF de rendimiento en producción	Métricas de la plataforma, k6
Registros (logs)	Una línea por petición: método, ruta, código y duración	morgan, pino, logging de Python
Respaldo de la base	Copia periódica y una restauración probada al menos una vez	Respaldo del proveedor, pg_dump programado
Límites del plan	Pausas por inactividad, cuotas y qué pasa al superarlas	Declarados en 2.9 como limitación

Rendimiento: se mide antes de optimizar

UNIDAD 5 • CADA MEJORA CON UN NÚMERO ANTES Y DESPUÉS

	ANTES		DESPUÉS
Lista sin paginar	GET /pedidos devuelve 3 000 filas en cada carga	→	Paginación por query (?pagina=&limite=20) y el total en la respuesta
Filtro sin índice	WHERE estado = ... recorre toda la tabla	→	CREATE INDEX sobre la columna de filtro; tiempo medido antes y después
Consultas en bucle (N+1)	Una consulta por cada pedido para traer su cliente	→	Un JOIN o la carga relacionada del ORM: una consulta en lugar de cien
Imágenes originales	Fotos de 4 MB servidas tal cual desde el almacenamiento	→	Redimensionadas al subir, en WebP, servidas con caché o CDN

Una optimización sin medición no se defiende: el tribunal pregunta "¿cuánto mejoró?".

BLOQUE 4 • DOCUMENTO

Documento final y manuales

Lo que falta del Capítulo 2, el Capítulo 3 y los anexos obligatorios.

04

Lo que falta del documento

SE ESCRIBE SOBRE EL MISMO ARCHIVO, QUE YA TIENE EL CAPÍTULO 1 Y LOS APARTADOS 2.2 A 2.6

Apartado	Qué contiene
2.7 Seguridad	Autenticación y sesiones, autorización por rol, validación, cifrado de credenciales y comunicaciones, gestión de secretos
2.8 Pruebas	Plan por niveles y tabla de casos con ID, escenario, esperado, obtenido y estado, con la evidencia referida
2.9 Despliegue	Plataforma, procedimiento, entornos y URL pública; en móvil, distribución e instalable. Limitaciones del plan
Capítulo 3	Una conclusión por objetivo específico, con grado de cumplimiento y evidencia; recomendaciones
Anexos	Manual de usuario, manual de instalación y despliegue, repositorio, URL o APK, diagramas legibles

De 30 a 40 páginas sin preliminares ni anexos. Resumen de 300 palabras como máximo.

Preliminares completos: portada, índices de contenido, tablas, figuras y anexos; numeración romana hasta el Capítulo 1.

Conclusiones: una por objetivo, con su evidencia

CAPÍTULO 3 • EL TRIBUNAL LAS CONTRASTA CONTRA EL CAPÍTULO 1

Objetivo específico	Conclusión débil	Conclusión que se defiende
OE-2 • Diseñar la arquitectura	"Se diseñó una arquitectura adecuada."	"Cumplido: monolito modular (Fig. 8), desplegado en Render y Neon; cubre RF-01 a RF-12."
OE-4 • Aplicar seguridad	"El sistema es seguro."	"Cumplido: JWT de 60 min, bcrypt y control de rol en 14 rutas; 6 casos de 403 aprobados (Tabla 19)."
OE-5 • Ejecutar pruebas	"Se realizaron pruebas satisfactorias."	"Parcial: 22 de 24 casos aprobados; CP-11 y CP-17 siguen abiertos y pasan a recomendaciones."
OE-6 • Desplegar el sistema	"El sistema fue desplegado."	"Cumplido: en su URL pública desde el 26/09, con monitoreo de disponibilidad activo."

Un objetivo cumplido a medias se declara así. El tribunal valora la precisión, no el optimismo.

Las recomendaciones salen de lo pendiente: mejoras funcionales, escalabilidad y condiciones de adopción.

Dos manuales, dos lectores distintos

MANUAL DE USUARIO

Para quien usa el sistema en su rol

Organizado por tareas, no por pantallas: "Registrar un pedido", "Cambiar su estado".

Cada tarea con pasos numerados y capturas de producción con datos ficticios.

Una sección por rol, y otra con preguntas frecuentes y mensajes de error.

MANUAL DE INSTALACIÓN Y DESPLIEGUE

Para quien tenga que levantarlo de nuevo

Requisitos y versiones, clonado, variables de entorno (nombres, nunca valores) y ejecución local.

Creación de la base con migraciones o script, y datos iniciales de prueba.

Despliegue en la plataforma paso a paso, y cómo verificar que quedó en línea.

Prueba de fuego: otra persona sigue el manual de instalación y el sistema levanta.

BLOQUE 5 • DEFENSA

Protocolo y rúbrica de la defensa técnica

T5 • del 12 al 15 de octubre, en la franja de su grupo. Doce minutos por persona.

05

Doce minutos por persona

SIETE DE DEMOSTRACIÓN Y CINCO DE PREGUNTAS • CON CRONÓMETRO



Una diapositiva como máximo para el problema: desde el minuto uno se muestra el sistema.

A los 7 minutos se corta la demostración, esté donde esté. Ensáyenla con cronómetro en la tutoría T4.

Rúbrica de la defensa técnica

100 PUNTOS • VALE EL 20 % DE LA NOTA DEL MÓDULO

Criterio	Puntos	Logro pleno
Sistema en producción	25	El flujo Must completo funciona en la URL pública (o el APK) con el usuario de prueba
Dominio técnico	30	Explica y ubica en el repositorio el código, el esquema y sus decisiones; hace un cambio simple si se le pide
Seguridad demostrada	15	Muestra en vivo el 401 y el 403, y explica dónde se hace cumplir el rol
Pruebas y evidencia	10	Muestra la tabla de pruebas y ejecuta al menos una prueba automatizada
Respuesta a preguntas	15	Responde con precisión, reconoce lo que falta y lo sostiene con el sistema abierto
Tiempo y claridad	5	Demostración dentro de los 7 minutos, en el orden del protocolo

REGLA DE LA DEFENSA

Quien no ejecuta ni explica su sistema, no aprueba la defensa

La falta de dominio técnico anula el componente, con independencia de quién haya escrito el código.
Usar IA está permitido; no entender lo que entregó, no.

Ocho preguntas que se hacen en la defensa

PREPAREN LA RESPUESTA CON EL SISTEMA ABIERTO, NO DE MEMORIA

01

¿Dónde se verifica el rol?

Muestre la ruta y el middleware o la política RLS que devuelve el 403

02

¿Qué pasa si el token vence?

Qué responde la API y qué hace el frontend con ese 401

03

¿Cómo guarda las contraseñas?

Algoritmo de hash y dónde se aplica en el código

04

¿Dónde están los secretos?

Variables de entorno en la plataforma; muestre el .env.example

05

¿Por qué este stack?

Justificación contra un requisito por su ID, no por preferencia

06

¿Cómo sabe que funciona?

Una fila de la tabla de pruebas y su evidencia

07

¿Qué pasa si se cae?

Monitoreo, respaldo de la base y cómo se vuelve a desplegar

08

¿Qué quedó pendiente?

Lo declarado como Won't have y en recomendaciones, sin esconderlo

Se puede pedir un cambio pequeño en vivo: un texto, una validación, un campo.

El día de la defensa



Sistema abierto 30 minutos antes

La capa gratuita despierta y la base no está pausada



Un usuario de prueba por rol

Para mostrar el 403 hacen falta dos cuentas



Pantalla compartida probada

Audio, cámara y pantalla funcionando antes de su turno



Una prueba automatizada lista

El comando para ejecutarla, ya escrito en la terminal



Repositorio y documento abiertos

En el esquema, el contrato de API y la tabla de pruebas



Cronómetro propio

Siete minutos de demostración, ensayados en T4

Se defiende el sistema entregado en el E4, en su franja de tutoría, del 12 al 15 de octubre.

Lo que se muestra es la URL o el APK que figura en la entrega.

CIERRE

Lo que hay que dejar listo

E3, E4, cuestionarios y defensa.

06

E3 · Checkpoint técnico

CIERRA EL SÁBADO 3 DE OCTUBRE A LAS 23:59 · CON EL CUESTIONARIO Q3

Pieza del E3	Qué se entrega
Autenticación y roles	Sesión que vence y autorización por rol verificada en el servidor en cada ruta protegida
Pruebas con evidencia	Tabla 2.8 con camino feliz y error de cada Must, casos de 401 y 403, y al menos una suite automatizada
Documento	Capítulos 1 y 2 completos en borrador (2.1 a 2.8), con lo observado en el E2 corregido
Sistema	Todos los Must en producción, con validación en cliente y en servidor
Repositorio	Commits en días distintos, pruebas en el repositorio y README al día

Un rol que solo se esconde en el frontend no está implementado.

Consigna y rúbrica completas en la tarea "E3 · Checkpoint técnico" de Moodle.

E4 · Documento y sistema final

CIERRA EL SÁBADO 10 DE OCTUBRE A LAS 23:59 · CON EL CUESTIONARIO Q4

Pieza del E4	Qué se entrega
Documento completo	Capítulos 1, 2 y 3, preliminares, bibliografía y anexos, conforme al formato institucional
Despliegue verificado	Sistema en su URL pública o APK, funcionando el día de la entrega con el usuario de prueba
Manuales	Manual de usuario y manual de instalación y despliegue, como anexos
Monitoreo y rendimiento	Monitoreo de disponibilidad activo y la medición del RNF de rendimiento en producción
Repositorio final	Historial progresivo, README completo, pruebas y evidencia; sin secretos

Lo que se entrega el sábado 10 es lo que se defiende del 12 al 15.

Consigna y rúbrica completas en la tarea "E4 · Documento y sistema final" de Moodle.

Lo que viene

DOS ENTREGAS, UNA SEMANA DE DEFENSAS Y NINGUNA PLENARIA MÁS

Cuándo	Qué	Para qué
28 sep – 1 oct	Tutoría T3 · en la franja de su grupo	Revisión del E2 con el sistema en vivo
Sábado 3 oct · 23:59	E3 · Checkpoint técnico + Cuestionario Q3	Roles, pruebas con evidencia, Capítulos 1 y 2
5 – 9 oct	Tutoría T4 · en la franja de su grupo	Revisión del E3, documento completo y ensayo de la defensa
Martes 6 oct	Sin actividad	Los Grupos 3 y 4 tienen su T4 el viernes 9, a las 17:00 y 18:00
Sábado 10 oct · 23:59	E4 · Documento y sistema final + Cuestionario Q4	Documento completo, despliegue verificado, manuales y anexos
12 – 15 oct · en su franja	T5 · Defensa técnica evaluada	7 min de demostración del sistema desplegado + 5 de preguntas

Nos vemos en su tutoría.

Esta fue la última plenaria · T3 sigue esta semana, en la franja de su grupo

Esta semana

Roles en el servidor y pruebas con evidencia. E3 y Q3, el sábado 3.

Del 12 al 15 de octubre

Defensa en su franja: 7 minutos de sistema en vivo y 5 de preguntas.

Lo que no se prueba, no se defiende.